

OCUPA

Sistema de Reserva de Salas

Documentação Técnica

Senac Joinville

Arquivo-fonte: reserva-salas.html

Sumário

- Sumário..... 2
- 1. Visão geral 3
 - 1.1 Principais capacidades..... 3
- 2. Stack tecnológica 3
- 3. Arquitetura 4
 - 3.1 Padrão de renderização..... 4
- 4. Modelo de dados..... 4
 - 4.1 Usuário (users)..... 4
 - 4.2 Catálogo — salas e recursos (catalog) 5
 - 4.3 Reserva (bookings) 5
- 5. Papéis e permissões 6
- 6. Telas e fluxos 6
 - 6.1 Login 6
 - 6.2 Solicitar sala (requisitante, coordenador, admin) 6
 - 6.3 Minhas solicitações 7
 - 6.4 Aprovações (coordenador, admin)..... 7
 - 6.5 Calendário do prédio (coordenador, admin)..... 7
 - 6.6 Salas e recursos (coordenador, admin) 7
 - 6.7 Usuários (admin) 7
 - 6.8 Painel público / Modo TV 7
- 7. Regras de negócio centrais..... 8
 - 7.1 Detecção de sobreposição de horário..... 8
 - 7.2 Séries recorrentes..... 8
 - 7.3 Capacidade da sala 8
 - 7.4 Desativação em vez de exclusão 8
- 8. Persistência de dados 8
- 9. Dados de demonstração (seed) 8
- 10. Limitações conhecidas e pontos de atenção..... 9
- 11. Glossário rápido de funções-chave 9

1. Visão geral

Ocupa é uma aplicação web para gestão de reservas de salas de aula e ambientes de um prédio (o cenário de exemplo usado no cadastro é o Senac Joinville). O sistema cobre todo o ciclo: solicitação de sala por professores/colaboradores, aprovação pela coordenação, detecção de conflitos de horário, cadastro de salas e recursos, cadastro de usuários e um painel público em modo TV para exibir a ocupação em tempo real no corredor.

A aplicação é entregue como um único arquivo HTML autocontido (`reserva-salas.html`), sem backend, servidor ou banco de dados próprio — toda a persistência é feita através de uma API de armazenamento chave-valor exposta ao navegador (`window.storage`), com dados compartilhados entre todos os usuários.

1.1 Principais capacidades

- Solicitação de sala pontual (uma data) ou recorrente (série semanal em dias fixos, ex.: toda terça e quinta).
- Fluxo de aprovação por coordenadores (escopo por andar) ou coordenação geral (acesso total).
- Detecção automática de conflito de horário na mesma sala, com opção de aprovar mesmo assim ou rejeitar apenas o dia conflitante.
- Aviso ("FYI") quando dois pedidos pendentes concorrem pelo mesmo horário/sala, antes mesmo de qualquer aprovação.
- Calendário do prédio por dia, agrupado por andar e sala, com destaque visual de sobreposições.
- Painel público ("modo TV"), sem necessidade de login, mostrando o que está acontecendo agora e a próxima atividade em cada sala.
- Cadastro de salas (capacidade, andar, descrição, recursos disponíveis) e de recursos (projetor, ar-condicionado, etc.).
- Cadastro de usuários com três papéis de acesso e desativação sem perda de histórico.

2. Stack tecnológica

Camada	Tecnologia	Observação
Estrutura	HTML5 único arquivo	Sem build step, sem bundler
Estilo	CSS puro (custom properties)	Tema escuro; fontes Google Fonts
Tipografia	Space Grotesk, IBM Plex Mono, Inter	Carregadas via <link> do Google Fonts
Lógica	JavaScript vanilla (ES6+)	Sem framework (não usa React/Vue)
Renderização	Re-render via innerHTML	Função <code>render()</code> reconstrói o DOM a cada mudança de estado
Persistência	<code>window.storage</code> (get/set assíncrono)	Chave-valor, com dado compartilhado (<code>shared=true</code>)
Roteamento	Estado interno (<code>St.view</code>)	Não usa URL/hash — é tudo client-side em memória

Não há dependências de terceiros em JavaScript (nenhum npm, nenhum CDN de framework) — apenas a folha de fontes do Google Fonts é externa.

3. Arquitetura

O arquivo é organizado em blocos comentados, na seguinte ordem, dentro da tag `<script>`:

Bloco	Responsabilidade
STATE	Objeto global St com todo o estado da aplicação em memória
UTIL	Funções auxiliares: datas, formatação, escape de HTML, checagem de sobreposição de horário
STORAGE	loadKey/saveUsers/saveCatalog/saveBookings — leitura e escrita em window.storage
SEED DATA	Dados de demonstração (usuários, salas, recursos, reservas) usados na primeira execução
DOMAIN HELPERS	Regras de negócio: salas visíveis por papel, agrupamento de pendências, cálculo de datas de série
ACTIONS	Funções expostas em window.* chamadas pelos eventos onclick/onsubmit do HTML gerado
RENDER: *	Uma função de renderização por tela (login, shell, solicitar, minhas, aprovações, calendário, salas, usuários, TV)
BOOT	Carrega dados do storage e dispara a primeira renderização

3.1 Padrão de renderização

O app segue um padrão simplificado de estado único e renderização declarativa, sem um framework:

- Toda a interface é uma função pura do objeto de estado St (dados de negócio) e St.transient (estado de UI: qual formulário está aberto, filtros, seleção de dias etc.).
- Qualquer ação do usuário (window.doLogin, window.submitRequest, window.decideGroup...) altera St e então chama render().
- render() decide entre três modos: painel TV (renderTV), tela de login (renderLogin, quando não há usuário logado) ou o shell autenticado (renderShell), que desenha a barra superior, as abas visíveis para o papel do usuário e o conteúdo da aba ativa.
- Não há Virtual DOM: cada render() substitui o innerHTML inteiro do container correspondente.

4. Modelo de dados

Três coleções são persistidas, cada uma sob uma chave própria em window.storage:

```
const KEYS = { users: "ocupa_users", catalog: "ocupa_catalog", bookings: "ocupa_bookings" };
```

4.1 Usuário (users)

Campo	Tipo	Descrição
id	string	Identificador único (ex.: u_renata, ou gerado por uid('user'))
name	string	Nome completo
email	string	E-mail institucional

role	enum	requisitante · coordenador · admin
floors	string[]	Andares sob responsabilidade (apenas para coordenador)
active	boolean	Usuário desativado não aparece na tela de login, mas mantém histórico

4.2 Catálogo — salas e recursos (catalog)

Sala (rooms):

Campo	Tipo	Descrição
id	string	Identificador único
name	string	Nome/número da sala
floor	string	Andar (ordenado por FLOOR_ORDER: Térreo, 1º a 4º andar; andares fora dessa lista vão para o fim, em ordem alfabética)
capacity	number	Capacidade máxima de pessoas
description	string	Texto livre exibido como dica ao solicitante
resourceIds	string[]	IDs de recursos disponíveis na sala
active	boolean	Sala desativada some das opções de nova reserva, mas mantém histórico

Recurso (resources): apenas { id, name } — ex.: Projetor, Ar-condicionado, Quadro branco, Computadores, Sistema de som, Tomadas extras.

4.3 Reserva (bookings)

Campo	Tipo	Descrição
id	string	Identificador único da instância (um dia específico)
seriesId	string null	Presente quando a reserva pertence a uma série recorrente; null para pedido pontual
roomId	string	Sala reservada
requesterId	string	Usuário que fez o pedido
title	string	Nome da aula/atividade
responsavel	string	Nome do responsável presencial (pode diferir do solicitante)
turmaSize	number	Quantidade de pessoas esperadas
date	string (YYYY-MM-DD)	Data da instância
startTime / endTime	string (HH:MM)	Horário da instância
status	enum	pending · approved · rejected · cancelled
createdAt	ISO datetime	Data/hora de criação do pedido
decidedBy / decidedAt	string / ISO datetime	Quem decidiu e quando (aprovação/rejeição)

Ponto importante do modelo: uma reserva recorrente não é armazenada como uma única regra — o sistema gera, no ato da solicitação, uma instância de booking por data que cai nos dias da semana escolhidos dentro do período informado. Todas as instâncias compartilham o mesmo `seriesId`, o que permite aprovar, rejeitar ou cancelar "a série toda" (ou o restante dela) de uma vez.

5. Papéis e permissões

Papel	Rótulo na tela	Pode solicitar sala	Pode aprovar	Escopo de aprovação	Gerencia salas/recursos	Gerencia usuários
requisitante	Professor(a) / colaborador(a)	Sim	Não	—	Não	Não
coordenador	Coordenador(a) delegado(a)	Sim	Sim	Somente andares em <code>user.floors</code>	Somente andares sob sua responsabilidade	Não
admin	Coordenação geral	Sim	Sim	Todos os andares	Todas as salas	Sim, cadastro completo

As abas visíveis na barra superior são montadas dinamicamente conforme o papel do usuário logado (função `renderShell`): requisitante vê apenas "Solicitar sala" e "Minhas solicitações"; coordenador e admin também veem "Aprovações" e "Calendário"; "Salas e recursos" aparece para coordenador e admin; "Usuários" é exclusiva do admin.

Importante: a restrição de coordenador ao aprovar é apenas para o que ele vê nas telas de Aprovações e Salas — a tela de Solicitar sala permite a qualquer papel (inclusive coordenador e admin) pedir reserva em qualquer sala do prédio, sem restrição de andar.

6. Telas e fluxos

6.1 Login

Tela inicial sem senha: lista os usuários ativos como botões ("seleção de perfil", modelo de demonstração/protótipo, não autenticação real). Também oferece o link "Abrir painel público", que leva ao modo TV sem exigir login.

6.2 Solicitar sala (requisitante, coordenador, admin)

Formulário para pedir uma sala. Campos: nome da aula/atividade, responsável (pré-preenchido com o nome do usuário logado), sala, quantidade de participantes, e o tipo de pedido:

- Pontual: uma única data + horário de início e fim.
- Recorrente: período (data inicial e final), dias da semana selecionáveis (chips Dom–Sáb) e horário único aplicado a todas as ocorrências.

Validações no envio (`submitRequest`): título obrigatório; quantidade de participantes não pode exceder a capacidade da sala; em pedidos pontuais, data e horários obrigatórios e horário final maior que o inicial; em

recorrentes, período válido (data final após a inicial), horário válido, ao menos um dia da semana marcado, e o período deve gerar ao menos uma data. Cada pedido nasce com status pending.

6.3 Minhas solicitações

Lista os pedidos do usuário logado, agrupados por série (ou por pedido avulso), com contagem de status (pendente/confirmada/rejeitada/cancelada) e, para séries, uma prévia das próximas datas com seus respectivos status. Permite cancelar um pedido pontual (se pendente ou aprovado) ou cancelar as próximas aulas restantes de uma série (as instâncias passadas não são alteradas).

6.4 Aprovações (coordenador, admin)

Lista os pedidos pendentes visíveis ao papel do usuário (todas as salas para admin; apenas salas dos andares do coordenador), agrupados por série. Para cada grupo, mostra:

- Um aviso "FYI" quando outro pedido pendente (de série diferente) concorre pelo mesmo horário/sala — visível antes de qualquer decisão.
- Alerta de capacidade quando a turma informada excede a capacidade da sala.
- Botões para aprovar todas as instâncias do grupo ou rejeitar todas de uma vez.

Ao aprovar, o sistema verifica, instância por instância, se já existe uma reserva aprovada com sobreposição de horário na mesma sala/data. As instâncias sem conflito são aprovadas imediatamente; as que colidem com algo já aprovado ficam pendentes e abrem uma caixa de conflito, permitindo, por instância: "Aprovar mesmo assim" (aceita a sobreposição) ou "Rejeitar este dia" (rejeita só aquela data, mantendo o restante da série).

6.5 Calendário do prédio (coordenador, admin)

Grade por data (seletor de data), organizada por andar e sala, mostrando todas as reservas do dia que não estão canceladas ou rejeitadas. Sobreposições de horário na mesma sala são destacadas visualmente com o marcador "⚠ conflito".

6.6 Salas e recursos (coordenador, admin)

Cadastro de tipos de recurso (lista simples de nomes, adição livre pelo admin) e cadastro de salas: nome, andar (com autocomplete via datalist, incluindo a ordem padrão de andares), capacidade, descrição e recursos disponíveis (checkboxes). Salas podem ser desativadas (saem das opções de nova reserva, mas o histórico é preservado) e reativadas. Coordenadores só veem/editam salas dos andares sob sua responsabilidade; o admin vê e edita todas.

6.7 Usuários (admin)

Cadastro de pessoas: nome, e-mail, papel (requisitante, coordenador ou admin) e, quando o papel é coordenador, os andares sob responsabilidade (checkboxes). Usuários podem ser desativados/reactivados sem perda do histórico de reservas já feito por eles.

6.8 Painel público / Modo TV

Tela cheia, sem exigir login, pensada para um monitor no corredor. Mostra relógio e data atuais, o turno (Manhã/Tarde/Noite conforme o horário), e um card por sala agrupado por andar, indicando:

- Se a sala está "AO VIVO" agora — com nome da atividade, responsável e horário; ou
- "— livre agora —", com a próxima atividade agendada, se houver.

O relógio é atualizado a cada segundo; a cada minuto (segundo 0), o painel recarrega as reservas do armazenamento para refletir mudanças feitas por outras sessões (outros usuários aprovando/criando pedidos em paralelo).

7. Regras de negócio centrais

7.1 Detecção de sobreposição de horário

```
function timeOverlap(s1,e1,s2,e2){ return s1<e2 && s2<e1; }
```

Duas reservas conflitam quando estão na mesma sala, na mesma data, e seus intervalos de horário se sobrepõem segundo essa regra. Ela é usada em três pontos distintos: ao decidir uma aprovação (contra reservas já aprovadas), ao exibir o aviso FYI entre pedidos pendentes concorrentes, e ao renderizar o calendário do dia (destaque visual).

7.2 Séries recorrentes

`generateSeriesDates()` percorre dia a dia o intervalo `[dataInicial, dataFinal]` e seleciona as datas cujo dia da semana está entre os marcados. Cada data gera uma instância de reserva independente, todas ligadas pelo mesmo `seriesId`. Isso significa que aprovação, rejeição e cancelamento podem agir sobre a série inteira, mas conflitos são resolvidos instância a instância — uma série pode terminar parcialmente aprovada.

7.3 Capacidade da sala

A quantidade de participantes informada no pedido é validada contra a capacidade da sala no momento do envio; além disso, a tela de Aprovações realça pedidos cuja turma excede a capacidade, mesmo que o valor tenha sido salvo (por exemplo, se a sala teve a capacidade reduzida depois do pedido).

7.4 Desativação em vez de exclusão

Usuários e salas nunca são excluídos — apenas marcados como `active=false`. Isso preserva a integridade do histórico de reservas (uma reserva antiga continua referenciando o mesmo `requesterId/roomId` mesmo que a pessoa ou a sala não estejam mais ativas).

8. Persistência de dados

Não há backend HTTP nem banco de dados tradicional. O app usa uma API de armazenamento assíncrona injetada no navegador, `window.storage`, com os métodos `get(key, shared)` e `set(key, value, shared)`. Todo o app usa `shared=true`, ou seja, os dados são compartilhados entre todos os usuários/sessões que abrirem o mesmo arquivo — é assim que uma aprovação feita por um coordenador aparece para o requisitante e no painel de TV.

```
await window.storage.get(KEY, true); await window.storage.set(KEY, JSON.stringify(valor), true);
```

Na inicialização (`boot()`), cada coleção é carregada por `loadKey()`: se a chave já existe no storage, os dados salvos são usados; caso contrário, dados de demonstração (seed) são gerados e gravados. Isso garante que a primeira execução já mostre o app funcionando (usuários, salas e reservas de exemplo, incluindo duas "ao vivo" no horário atual para o painel de TV nunca abrir vazio).

Falhas de leitura ou escrita no storage são tratadas com `try/catch` e avisadas ao usuário via toast ("Não foi possível salvar..."), sem travar a interface.

9. Dados de demonstração (seed)

Ao rodar pela primeira vez (storage vazio), o app popula:

- 4 usuários: Renata Alves (admin), Carlos Menezes (coordenador dos andares 2º e 3º), Fernanda Lima e João Pedro Rocha (requisitantes).
- 6 salas em 3 andares (Térreo, 2º e 3º), incluindo um Auditório, e 6 tipos de recurso.
- Reservas de exemplo: duas "ao vivo" (uma delas terminando/começando perto do horário atual, para o painel de TV sempre ter algo a mostrar), uma série aprovada recorrente (terças e quintas, 19h–22h, abrangendo passado e futuro), dois pedidos pendentes conflitantes para o dia seguinte (demonstra a detecção de conflito) e um pedido pendente sem conflito (demonstra aprovação limpa).

10. Limitações conhecidas e pontos de atenção

- Não há autenticação real: o "login" é apenas a seleção de um perfil da lista de usuários ativos, sem senha ou verificação de identidade.
- Toda a lógica roda no cliente (navegador); não há validação de servidor, então as regras de negócio dependem inteiramente do JavaScript carregado (não deve ser considerado seguro contra um usuário mal-intencionado com acesso ao console).
- last-write-wins: como a escrita no storage não usa bloqueio/transação, duas edições simultâneas na mesma chave podem se sobrescrever.
- O relatório de conflitos em Aprovações compara apenas contra reservas já aprovadas no momento da decisão; o aviso FYI entre pendentes é informativo e não impede a aprovação simultânea de ambos os lados.
- Sem paginação: todas as listas (usuários, salas, reservas) são renderizadas por completo a cada render(), o que pode ficar custoso com um volume de dados muito grande.
- O calendário e as aprovações mostram apenas o dia selecionado / pendências atuais — não há uma visão de mês ou exportação de agenda.

11. Glossário rápido de funções-chave

Função	O que faz
boot()	Carrega usuários, catálogo e reservas do storage (ou gera seed) e chama render().
render()	Decide entre TV, login ou shell autenticado e redesenha o DOM.
submitRequest()	Valida e grava um novo pedido (pontual ou série recorrente).
decideGroup() / rejectGroup()	Aprova ou rejeita todas as instâncias pendentes de um grupo (pedido avulso ou série).
forceApprove() / rejectInstance()	Resolve uma instância específica em conflito, mantendo o restante do grupo intacto.
conflictsForInstance()	Retorna as reservas que colidem em horário com uma instância, filtrando por status.
generateSeriesDates()	Gera a lista de datas de uma série a partir de período + dias da semana.
visibleRoomsFor()	Filtra as salas visíveis conforme o papel do usuário (admin: todas; coordenador: só seus andares).

pendingGroupsFor() / myGroupsFor()	Agrupam reservas por série para as telas de Aprovações e Minhas solicitações.
attachTVTimer()	Atualiza o relógio do painel de TV a cada segundo e recarrega reservas a cada minuto.